AI Native Design for Product Designers

Full Course Handout

All Sessions 1–5 · v9 · Oct 18, 2026

© 2026 Ladan Sojudi.
Licensed under Creative Commons BY-NC-ND 4.0: you may share it unchanged, with credit, for non-commercial use.
creativecommons.org/licenses/by-nc-nd/4.0/

Contents

Session 1

Low-usage tips for working with Claude and ChatGPT

Session 2

Skills and Plugins — Session 2

Session 3

Skills and Plugins — Session 3

Session 4

Session 5

Skills and Plugins — Session 5

Appendix — Templates and Skill Files

© 2026 Ladan Sojudi

Session 1

This handout summarizes what you should take with you from Session 1.

1. Main points

- AI can help in the design process, but the final decision belongs to the designer.
- An answer that sounds convincing (Plausible) is not necessarily checked (Verified).
- Ideas, interface text and drafts are easier to judge.
- Numbers, statistics, research findings, facts, quotes, dates, names, accessibility rules and legal claims need more checking.
- One question helps: if the AI is wrong, will I notice quickly?
- We choose the tool and the model based on the task.

2. Checking an answer: Claim → Source → Support

- **Claim:** what exactly was said?
- **Source:** where is the original source?
- **Support:** does the source really say this?
- **Relevance:** does it apply to my situation?
- Having a source is not enough; the source must support the claim.

3. Context, the box and Tokens

- **Context:** the information the AI has right now to answer: our instruction, the earlier chat, files, project information and tool results.
- **Context Window:** a box of limited size that all of this, and the AI's own answer, must fit into.
- **Token:** the pieces the box is filled with. A word is not always one Token.
- Plan usage is also counted in Tokens.
- Relevant information matters more than more information (Relevant Context > More Context).
- **Two pages to see for yourself:**
  - Context Window size of Claude models: https://support.claude.com/en/articles/8606394-how-large-is-the-context-window-on-paid-claude-plans
  - A simple explanation of Tokens from OpenAI: https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them
- Do not memorize the numbers; they keep changing.

© 2026 Ladan Sojudi

4. How to use fewer Tokens

- **What uses more:** a long chat, a big file, a stronger model and higher Effort, tools like web search, and multi-step tasks.
- **How to use less:**
  - One chat per task; when the task changes, open a new chat.
  - First think about what you need, then ask once with all the information needed.
  - Send related questions in one message.
  - Read your message once before sending.
  - Refer to an earlier point in the same chat instead of typing it again.
  - Give only the file, or the part of it, that this task needs.
  - Put files you need several times in a Project.
  - Say how long the answer should be.
  - Choose the model and Effort to fit the task.
  - Turn off tools you do not need, like web search and connections to other apps. Turn off "Extended thinking" too for routine work.
  - For a quick question, web search is enough; keep Research mode for broad tasks.
  - A very long chat gets summarized after a point and takes a bigger share of the limit; for new work, open a new chat.
- **Plan limit:** paid Claude plans have a five-hour limit and a weekly limit. Chat, Claude Code and the desktop app share one limit.
- **Where to see it:** in Claude, Settings and then Usage.
- **In ChatGPT:** use search for quick facts and keep Deep research for questions that need depth. A counter inside the product shows how many Deep research tasks you have left.

5. Before you upload

- **Three questions:**
  - Does the AI really need this information for this task?
  - Am I allowed to share this information?
  - Has your organization or employer approved this tool?
- Remove or anonymize names, phone numbers, emails and private information you do not need before uploading.
- **Settings of a personal Claude account (Free, Pro and Max):**
  - In Settings and then Privacy there is the option "Help improve our AI models". If it is on, chats may be used to train models. If you turn it off, they are not used from then on.
  - Work accounts have separate rules; ask your organization.

© 2026 Ladan Sojudi

- A simple rule: share only what is needed and what you are allowed to share.

6. A better instruction: six parts

- **A good instruction has these six parts, in this order:**
  1. Say what role the AI has in this task.
  2. Say what the task is about: what the product is, who its user is, and how it works now.
  3. Put in your evidence and files, each one separate; and say what you still have no evidence for.
  4. Say exactly what you want the AI to do, and why. If the order matters, number the steps.
  5. Give the rules with their reasons, and write "do this", not "don't do that".
  6. Say what the answer should look like; if you have an example, include it.
- **Three rules we almost always add to part 5:**
  - If you do not know something, say you do not know.
  - For each claim, say where it comes from.
  - Use only the information I gave you.
- **Why this order:** information and evidence on top, the request at the bottom. Anthropic's guide suggests the same: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Empty template (copy it and fill in the brackets):

© 2026 Ladan Sojudi

```
ROLE
[One sentence: who AI should be for this job.]

ABOUT THE PRODUCT AND ITS USERS
[What the product is, who uses it, and how it works today.]

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
[Your files, data, user quotes or research, each one separate.
What we have no evidence for yet.]

WHAT I WANT YOU TO DO AND WHY
[The exact job, and why you need it.
Number the steps if the order matters.]

RULES AND WHY
- If you do not know something, say so.
- For each claim, say where it comes from.
- Use only the information I gave you.
[- Your own rules, each with a reason.]

FORMAT
[What the answer should look like. Add an example if you have one.]
```

The example we saw in class:

```
ROLE
You are a UX writer.

ABOUT THE PRODUCT AND ITS USERS
Loop is a habit app for adults recovering from burnout.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
Users say: "The app feels like another job."
We do not know which reminder wording works.

WHAT I WANT YOU TO DO AND WHY
Write 3 short daily reminders, so we can test them.

RULES AND WHY
Use only this information. Say when you do not know,
and label any guess, so we can check it.

FORMAT
A table: reminder | tone | based on
```

The weak instruction, by contrast, was: "Write a reminder notification for a habit app". The AI had to guess the user, the tone and the evidence.

© 2026 Ladan Sojudi

7. Next session

We start from a short Brief: the client's request. We know some things and we do not know others; we find what is missing with three research methods.

Sources

- **Context and tokens (section 3):** https://support.claude.com/en/articles/8606394-how-large-is-the-context-window-on-paid-claude-plans and https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them
- **Using fewer tokens (section 4):** https://support.claude.com/en/articles/9797557-usage-limit-best-practices, https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work, https://support.claude.com/en/articles/11095361-when-should-i-use-web-search-extended-thinking-and-research and https://help.openai.com/en/articles/10500283-deep-research-in-chatgpt
- **Before you upload (section 5):** https://privacy.claude.com/en/articles/12109829-how-do-i-change-my-model-improvement-privacy-settings and https://privacy.claude.com/en/articles/10023548-how-long-do-you-store-my-data
- **A better instruction (section 6):** https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

© 2026 Ladan Sojudi

Low-usage tips for working with Claude and ChatGPT

This sheet gives practical tips so your usage limit does not run out too soon. All tips are taken from the official Anthropic and OpenAI pages, and the source number is next to each tip; the full list of links is at the end of the sheet.

Why tokens matter

- In paid Claude plans there is a "session" limit that resets every five hours and a weekly limit; usage of claude.ai, the desktop app and Claude Code all counts against this one allowance. [2]
- According to Anthropic's guide, these raise usage: message length, attached file size, the length of the same chat, tools like Research and web search, model choice, effort level, creating Artifacts, and multi-step tasks (running code, creating files, browsing a website). [1]
- A long chat costs more: the Claude Code docs say the whole chat is sent again with every request, so even a one-line question in a chat that has been open since morning counts for the whole history (at a cached rate). [9]
- When a chat nears the context limit, "automatic context management" summarizes old messages; the chat continues but these long chats take a bigger share of the usage limit. [2]
- ChatGPT follows the same logic: model choice, input and output size, reasoning level, Fast mode and multi-step tasks raise Work usage. [12]
- Background (for understanding only): Projects content in Claude is cached, and cached content costs less of the allowance when used again; but the cache expires after some idle time, and coming back after a long break counts all the content again. [1]

In Claude

Chat and prompt

- Start a new chat for every unrelated task so Claude does not carry extra context; and when a long chat nears the limit, open a new one. [2] [3]
- Plan before you start: ask related questions together and give the context from the beginning. [1]
- Write precisely and briefly; a vague request adds back-and-forth. Review the prompt once before sending to reduce follow-up messages. [1]
- Batch similar tasks into one message, and inside the same chat refer to what you said earlier ("the one I mentioned above") instead of repeating the text. [1]
- If you only need a quick answer, say clearly that you do not want a file or a full output. [3]

Model, effort and Thinking

© 2026 Ladan Sojudi

- A higher effort level uses more tokens; Low and Medium suit routine tasks and keep more of your allowance. [7]
- For routine tasks turn extended thinking off (on models where it can be turned off) and lower the effort. [2] [7]
- Stronger models use more; the Claude Code docs recommend Sonnet for most tasks and Opus only for complex decisions. [2] [9]

Projects and files

- Put reference documents you need all the time in a Project so cached content is used. [1]
- Keep the Project instructions short and delete files that are no longer needed. [2]

Skills, Plugins and tools

- Skills work with "progressive disclosure": Claude first decides which one is relevant and loads only that one; unlike a Project's background knowledge, which is always loaded into the chat. [5]
- The Claude Code docs say to move specialized instructions from an always-loaded file into a Skill, because a Skill loads only when needed; and keep that base file under 200 lines. [9]
- Turn off connected apps and web search when you do not need them; tools use many tokens. [2]

Cowork, Research and Claude in Chrome modes

- Multi-step tasks (running code, creating files, connected apps, the browser) use more because every step costs tokens; gather related work into one task. [3]
- Auto mode in Cowork uses more of the allowance than the other modes. [3]
- Research mode is for broad collection (five or more tool calls, 1 to 3 minutes); for quick, fact-based questions, web search with one or two calls is enough. [6]
- Claude in Chrome tasks are heavier than a normal chat and long tasks use more; break a complex task into smaller steps and close unnecessary tabs. "Automatically approve" mode also does extra safety checks and uses more. [8]
- See your usage in Settings > Usage; everything you do with Claude counts against your plan allowance. [1] [3] [4]

In ChatGPT

- For quick questions or short chats, normal chat or search is faster; use Deep research only for questions that need depth and completeness. [10]
- Write the Deep research prompt precisely: the question, the desired output and the limits. Review and edit its proposed plan, and limit sources to specific domains with Sites > Manage sites. [10]
- A counter inside the product shows how many Deep research tasks you have left; on plans with a monthly allowance it resets 30 days after first use. [10]
- For everyday questions, Instant mode gives fast responses. [11]

© 2026 Ladan Sojudi

- In Work: before a big task check Settings → Usage, choose the model and reasoning level up front and start with low effort; raising effort does not always give a better result and does not replace a clear instruction or the files you need. [12]
- Use Fast mode selectively because it uses more of the allowance; switching models also does not give back the shared allowance. [12]
- In Settings > Usage & billing, the "Top consuming chats" list shows the heaviest Work and Codex chats of the past month (regular Chat is not included). [13]
- In Projects, reuse one Project for recurring work, remove older or unnecessary files or combine their content, and use project-only memory so chats do not reference conversations outside the Project. [14]
- Press "New chat" whenever you want a fresh start. [16]
- Be clear and unambiguous in the prompt, give enough context, and if the answer is not good, rewrite or simplify the prompt. [15]
- The old ChatGPT agent mode is no longer available and multi-step tasks moved to ChatGPT Work; the old page says each agent request counts against the monthly limit, including requests from scheduled tasks, so stop scheduled tasks you do not use. [17]

Habits of this course

- **One task, one chat:** the same as the official advice "a new chat for unrelated work"; stale context costs again with every message. [2] [3] [9]
- **Only short, approved .md files in Context:** in line with "short instructions, delete unneeded files" in Projects and ChatGPT's advice to delete unnecessary uploads. [2] [14]
    - Keep drafts and old versions outside Context.
- **A state.md file instead of a long history:** a long chat costs more and automatic summarizing uses usage too; a short summary of where the work stands keeps starting a fresh chat cheap. [2] [9]
    - Note: "write a summary and then open a new chat" is a habit of this course; the official source only recommends fresh, short context.
- **Checking in a separate chat:** do the check in a fresh chat so the long history of the main work does not come along. [3] [9]
- **Model and effort that fit the task:** simple work with low effort; Thinking and Research only when really needed. [2] [6] [7]

Sources

- [1] https://support.claude.com/en/articles/9797557-usage-limit-best-practices
- [2] https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work
- [3] https://support.claude.com/en/articles/13345190-get-started-with-claude-cowork

© 2026 Ladan Sojudi

- [4] https://support.claude.com/en/articles/16761823-claude-cowork-and-chat-are-one-claude
- [5] https://support.claude.com/en/articles/12512176-what-are-skills
- [6] https://support.claude.com/en/articles/11095361-when-should-i-use-web-search-extended-thinking-and-research
- [7] https://support.claude.com/en/articles/8664678-change-the-model-effort-and-thinking-settings
- [8] https://support.claude.com/en/articles/12902405-claude-in-chrome-troubleshooting
- [9] https://code.claude.com/docs/en/costs
- [10] https://help.openai.com/en/articles/10500283-deep-research-in-chatgpt
- [11] https://help.openai.com/en/articles/11909943-gpt-5-in-chatgpt
- [12] https://help.openai.com/en/articles/20001516-managing-usage-with-gpt-6-astra-in-work-and-codex
- [13] https://help.openai.com/en/articles/20001478-reviewing-work-and-codex-usage-and-using-personal-analytics-in-chatgpt-desktop
- [14] https://help.openai.com/en/articles/10169521-using-projects-in-chatgpt
- [15] https://help.openai.com/en/articles/10032626-prompt-engineering-best-practices-for-chatgpt
- [16] https://help.openai.com/en/articles/12677804-what-is-chatgpt-faq
- [17] https://help.openai.com/en/articles/11752874-chatgpt-agent

© 2026 Ladan Sojudi

Session 2

This handout summarizes what you should take with you from Session 2. The goal of the session was to understand where the problem really is, not to design anything.

1. Main points

- In a real project the client rarely gives a complete Brief. We start with only a short email.
- First we ask: what do we know, and what is missing? The missing parts are gaps. We ask the client about the gaps, and what the client cannot answer stays unknown.
- Then the AI writes a short Brief from the email and the answers. It keeps what we know (K1, K2 ...) apart from what we do not know, and turns the unknowns into research questions (RQ1, RQ2 ...).
- We check the Brief ourselves and approve it. Then we use three research methods to answer the research questions.

2. Writing the Brief

- **What it is:** before the Project exists, we write the Brief in a normal chat. It keeps what we know apart from what we do not know.
- **Why:** a client rarely gives a complete Brief on day one. If we ask the AI to "write a Brief" from a short email, it fills the holes with guesses.

Step 1 — What we have. The whole email from the client of QuickBook, a beauty salon:

```
- The salon has three branches.
- Some customers book an appointment but do not come (no-show).
- Receptionists spend a lot of time on booking calls.
- We think we may need something like Calendly.
- Please design a better booking experience for us.
```

Step 2 — The gaps. What is missing is a gap. We ask the client about the gaps (in class we assume the answers came back):

- How do customers book today? Mostly by phone and WhatsApp.
- How many no-shows are there, and why? There is no reliable data.
- Some things stay unknown until the research. That is normal; the client will not do our research.

Step 3 — The prompt. Open a new chat, put the email and the answers in the matching places, and send this. It uses the six parts from Session 1: what we know and what we do not know are in separate places, and the AI writes one Brief only.

© 2026 Ladan Sojudi

```
ROLE
You are a UX researcher helping a product design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
What we know (the client's email and the answers):
[paste the email and the answers here]
What we do not know:
[what the client could not answer, for example:
how many no-shows there are, and why]

WHAT I WANT YOU TO DO AND WHY
Write one short Discovery Brief for QuickBook,
because we need to see what we know and what we do not know
before we research.
Sections: Known facts, Unknowns, Research questions.
Turn each unknown into a research question.
Give each known fact an ID (K1, K2 …)
and each research question an ID (RQ1, RQ2 …).

RULES AND WHY
- Every known fact must come from the email or the answers,
  so we can check it.
- Keep known facts and unknowns separate,
  so I can see how sure we are.
- If information is missing, keep it as unknown, not a guess.
- Do not suggest features or UI,
  because we have not decided what to build.
- Keep these IDs stable in every later output,
  because later files refer to them.

FORMAT
One Brief, short. Stop after writing it.
```

Step 4 — Check and save. Read the Brief yourself and check it. Fix anything the AI wrote as a fact that the client never said. When you approve it, save it as `Discovery Brief v2.md`, and keep it for the next section.

3. How we work: a Project in Claude

- We create a Project named QuickBook. All chats of the course happen inside this Project.
- When you create the Project, Claude asks "What are you trying to achieve?". This field is the goal of the whole Project in one or two sentences: what the work is for. It is not the rules (the rules go in Instructions, in the next step). Claude needs it before it creates the Project. So for this course we write the goal of our own project, like this:

© 2026 Ladan Sojudi

```
Discovery for QuickBook, a booking app for a service business:
understand the problem and its users from research and evidence,
before deciding what to build.
```

- After the Project exists, you add the Project instructions. A Project has two separate places: Instructions for the rules, and Context for the files. Instructions are the standing rules Claude reads at the start of every chat in this Project: who Claude is in this project (ROLE), which files to read first, and the rules with their reasons (RULES AND WHY). They hold no research data; the files go in Context. Paste the Instructions text below into it; this is the Instructions text of this course. For your own other projects, an empty version is at the end of this handout (Appendix: Project instructions template).
- The Project instructions text of this course (copy it):

```
ROLE
You are helping a product designer with the QuickBook
discovery project. Before any work, read README.md and state.md,
because they hold our rules and where the project is now.

RULES AND WHY
- Keep evidence, interpretation, guesses and unknowns separate,
  so I always know how sure we are.
- Write a guess as a guess, never as a fact.
- Say "I don't know" when our files do not answer something.
- Use only data, numbers, quotes and participants that exist
  in our files, because invented ones would mislead the design.
- Give a source for every important claim, so I can check it.
- Treat external research as evidence about other people,
  not about QuickBook users.
- Use our IDs (K1, RQ1 …) exactly as they are,
  because every file refers to them.
- Stay with understanding the problem: give UI or feature ideas
  only when I ask, because we have not decided what to build.
- Open only the files this task needs, and quote them by ID
  instead of copying them back, because long context uses
  more of my usage limit.
- Keep answers as short as the task allows, and ask before
  writing anything longer than one page, for the same reason.
```

- We put three files in Context: "Discovery Brief v2", `README.md` and `state.md` (templates in the Appendix). `README.md` holds the project rules and `state.md` says where we are now.
- Each new task is a separate chat in this Project, with a numbered name, like "01 — Research Plan v1". Each chat sees only the Instructions and the Context.
- The "Parking Lot" chat is the place for unrelated ideas, so the main chat stays short and the idea does not get lost.
- We save every approved output ourselves and add it to Context. When a new file arrives, we delete its old version.

© 2026 Ladan Sojudi

- Add only approved files to Context. Drafts stay in the chat.
- Do not add names, phone numbers and full interview texts to Context. Whatever is there, the AI sees.
- Empty templates of `README.md` and `state.md` are at the end of this handout.

4. Research plan

> **Where we are** the Project is ready and the approved Brief is in Context. Now we decide how to answer its research questions (RQ).

- **What it is:** a short table that says which method answers which research question (RQ).
- **Why we do it:** we choose the methods on purpose, not by habit.
- **How we do it:** with this prompt, in the chat "01 — Research Plan v1". The AI suggests which methods fit each research question. Then we choose the methods ourselves.

Research plan

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
"Discovery Brief v2": what we know (the K IDs)
and what we do not know yet (the RQ IDs).

WHAT I WANT YOU TO DO AND WHY
Make a lightweight research plan.
Suggest the research methods that fit each question.
A question can need more than one method.
For each method, say in one line why it fits.
Why: I want to choose methods on purpose, not by habit.

RULES AND WHY
- Stop after the plan and wait for me,
  because I choose the methods before any research starts.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
One row per research question:
what we want to learn · method · input · expected output · why this method
```

Our choice for this course. We chose three methods:

© 2026 Ladan Sojudi

- **Desk research:** to learn what is already known outside our project, so we know what to listen for in the interviews.
- **Competitive analysis:** to see how one real booking tool handles booking, rescheduling and cancelling.
- **Interviews:** the only one of the three that tells us about our own users.

If none of the three can answer a question, we write that down, so the gap stays visible.

5. Desk research

> **Where we are** the research plan says which method answers which question. We start with desk research.

- **What it is:** looking for what is already known outside our project, only about the things we do not know.
- **Why we do it:** to know what to listen for in the interviews. It is evidence about other people, never about QuickBook users.
- **How we do it:** with this prompt, in the chat "02 — Desk Research v1". Every finding must carry the exact sentence from its source.

Desk research

© 2026 Ladan Sojudi

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
"Discovery Brief v2". We have no data of our own yet.

WHAT I WANT YOU TO DO AND WHY
Do desk research for the research questions in the Brief.
Why: I want to know what is already known outside our project,
so I know what to listen for in interviews.

RULES AND WHY
- Cover only the questions our research plan gives to desk research,
  because the others belong to the competitive analysis or the interviews.
- Use at most 3 strong sources per research question,
  because a few strong sources are easier to check than many weak ones.
- Give each source an ID (S1, S2 …) and each finding an ID (E01, E02 …),
  so later files can point to them.
- For each source, give its type, year, country and setting,
  because a finding from a hospital in another country may not fit us.
- Label every finding as external, because it is evidence
  about other people, not about QuickBook users.
- Quote the exact sentence from the source. If you could not
  capture it, say so and mark any paraphrase as a paraphrase,
  because I will check every quote against its source.
- Say "I don't know" where you found no good source.
- Stay with findings and leave out UI or feature ideas,
  because we have not decided what to build.

FORMAT
For every finding: ID · RQ · finding · exact sentence · source ID · link
```

Check and save. The chat that wrote the research checks it badly, so we check it in a separate chat ("03 — Review") with a skill.

- **What a skill is:** an instruction we write once and use whenever we need it, instead of typing it again. `check-skill` is our own: it checks every claim against its source and flags what the source does not support. Its text is in the Appendix, and the `.zip` file is given separately.
- **How we check:** in the review chat, paste the research and say "check this". The skill labels each claim (Verified, Partially supported, Not supported, Not relevant) and does not rewrite anything.
- **Save:** only after the check and your own reading, ask for the final text in a code block and save it as `Desk Research v1.md`, and add it to Context.
- **Update the files:** a second skill, `update-files-skill`, writes the new rows for `state.md` (Status and History) after each approved file. Say "update project files" and save the new `state.md` the same way. Its text is in the Appendix.

© 2026 Ladan Sojudi

- **If you cannot build a skill** (for any reason): to build a skill, use the `.zip` file. Without a skill, use the `.md` file instead: add it to Context (or the Project's sources) and tell the AI "check this with the file `check-skill.md`". If even that is not possible, save the prompt on your computer and copy and paste it into the chat each time. The same goes for `update-files-skill`.

From now on, every section ends the same way: check with `check-skill`, save the approved file, add it to Context, update the project files.

6. Competitive analysis

> **Where we are** desk research told us what is already known outside our project. Next we look at one real booking tool.

- **What it is:** going through one existing booking tool (Calendly) as a customer and writing down only what is on the screens. What was not seen is written as "not seen".
- **Why we do it:** to see how a booking tool handles booking, rescheduling and cancelling, so we can ask better interview questions. It is one tool, not evidence about QuickBook users.
- **Privacy first:** Claude in Chrome takes screenshots of your active browser tab and, according to Anthropic, cannot filter sensitive content out of what it sees. Admins of Team and Enterprise plans can restrict or switch it off, and Anthropic advises against using it with work accounts that hold sensitive company data. Check your company's privacy rules before using it.
- **How we do it:** first go through the product yourself and take the screenshots of booking, rescheduling and cancelling your own test appointment (this is the best way), then attach them to the chat together with the first prompt. The prompts, in order: the walkthrough of your screenshots, the user flow and information architecture, then the analysis.

Step by step with screenshots

© 2026 Ladan Sojudi

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
Screenshots I took myself, in order, while booking one appointment, then rescheduling
and cancelling it
on my own Calendly booking page: [attach your screenshots]
The page link: [your own booking link]
The link from the confirmation email: [link]
We do not know yet how a booking tool handles booking,
rescheduling and cancelling, step by step.

WHAT I WANT YOU TO DO AND WHY
Go through my screenshots one by one. For each screenshot:
- give the screen an ID (SC01, SC02 …), so we can point to it later
- list what is on it: content, choices, buttons
- note what the customer has to decide here
Why: I want to see how an existing booking tool works,
so I can ask better questions in our interviews.

RULES AND WHY
- Describe only what is on the screenshots, because that is our evidence.
- If a step is missing between two screenshots, say "not seen"
  and do not guess it.

FORMAT
One block per screen: ID · what is on it · what the customer decides
```

User flow and information architecture

- **What it is:** a drawing of the booking tool made from your screenshots. The user flow shows the steps and decisions a customer goes through. The information architecture shows how the pages and content are grouped.
- **Why we do it:** a drawing shows the steps, the decisions and the structure at one glance, so we can compare them with our research questions and see what the tool does and does not cover.
- **How we do it:** open a new chat, attach the screenshots again, and ask the AI to draw the user flow and the information architecture in FigJam. This needs the Figma connection. It draws only what is on the screenshots.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
My screenshots, attached again: [attach your screenshots]. Screen IDs: SC01 onwards.

WHAT I WANT YOU TO DO AND WHY
Draw:
1. The user flow: every step and decision
   for booking, rescheduling and cancelling.
2. The information architecture:
   how pages and content are grouped.
Why: a drawing shows the steps, the decisions and the structure
at one glance, so we can compare them with our questions.

RULES AND WHY
- Draw only what you saw, because a guessed step looks just as real.
- If a path exists but you did not open it, mark it "not seen".
- Put the screen IDs on every step, so I can check each one.

FORMAT
Draw both in FigJam: the user flow as a flowchart, the information architecture as a
tree of pages.
```

Analysis (competitive)

- **What it is:** a short analysis of what the booking tool (Calendly) can and cannot tell us about our research questions that it can touch.
- **Why we do it:** one tool is not evidence about QuickBook users. The analysis shows what we learned from the tool, what it cannot tell us, and which questions to take to the interviews.
- **How we do it:** in the same chat, after the flow is drawn. For each research question the AI writes what the tool shows (with screen IDs as evidence), what it does not show, and questions for the interviews. Then we check it in "03 — Review" and approve it as "Competitive Analysis v1".

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
"Discovery Brief v2", the screens and the flow above.

WHAT I WANT YOU TO DO AND WHY
Analyse this flow against the research questions in the Brief that this tool can help
with.
For each RQ:
- what Calendly shows, with screen IDs as evidence
- what it does not show, or cannot tell us
- questions to take into our interviews
Why: I want to know what this tool can tell us about our questions,
and what it cannot.

RULES AND WHY
- Treat this as one tool, not as evidence about
  QuickBook users or what they need.
- Keep what you saw separate from what you think it means,
  so I know which parts to check.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.
- Stay with analysis and leave out UI or feature ideas for QuickBook,
  because we have not decided what to build.

FORMAT
One section per RQ: shows · does not show · questions for interviews
```

Check and save. Check the output in "03 — Review" with `check-skill`, read it yourself, and only then approve it. Save it as `Competitive Analysis v1.md`, add it to Context, and run `update-files-skill`.

7. Interviews

> **Where we are** desk research and one tool tell us nothing certain about our own users. Now we ask real people.

- **What it is:** talking to real users. For each research question in our Brief we want at least one interview question that answers it. A leading question is one that pushes the answer; instead we ask "How did you book the last time?".
- **Why we do it:** external research and one tool cannot tell us about our own users. Interviews can.
- **In class:** the interviews are simulated and are practice data. The AI can play the role of a user, but it is never evidence about a real user.

© 2026 Ladan Sojudi

- **How we do it:** first the questions, then the interview sheets, then (after the interviews) the analysis. After the interviews each pattern gets an ID and must say which participant it came from.
- The approved analysis is saved as "Interview Synthesis v1".

Questions

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
"Discovery Brief v2"
"Desk Research v1"
"Competitive Analysis v1"
None of these is evidence about our own users yet.

WHAT I WANT YOU TO DO AND WHY
Write interview questions for two groups:
1. Customers
2. Receptionists
Why: external research and one tool cannot tell us
about our own users. Interviews can.

RULES AND WHY
- Ask about real past behaviour before opinions,
  because people describe what they did more accurately
  than what they would do.
- Ask open, neutral questions that name no solution or tool,
  because a named solution pushes the answer.
- Treat desk research and Calendly as ideas to test,
  not as facts about our users.
- Ask about what we do not know, not about what we think we know,
  so the interviews can surprise us.
- Write at most 10 questions per group, so an interview fits its time.
- Show the questions only and wait,
  because I approve the questions before we make sheets.

FORMAT
Next to each question, the research question it serves.
At the end, any research question that no question covers.
```

Sheets

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The questions above. They are approved.

WHAT I WANT YOU TO DO AND WHY
Create two interview sheet templates:
one for customers, one for receptionists.
Why: during an interview I have to listen, ask and take notes at once.
A sheet makes that easier, and every interview gets recorded the same way.

RULES AND WHY
- Keep the approved questions, in the same order and with no new ones,
  because they are already approved.
- Use the participant ID instead of a name, phone or email,
  because we keep no personal details.

FORMAT
Top of each sheet: participant ID, role, date, method.
Moderator part: how to open, and common mistakes to avoid.
For each question:
- what to listen for, and one follow-up question
- quick-check options (for me only, not to read aloud)
- space for notes and an exact quote
- a "not asked" box
End of each sheet: an evaluation table, one row per research question,
with columns: answered yes / partly / no, and a note.
```

Analysis (interviews)

© 2026 Ladan Sojudi

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The sheets of C01, C02, C03 (customers), R01, R02 (receptionists).
This is simulated classroom data, not real research.
"Desk Research v1"
"Competitive Analysis v1"

WHAT I WANT YOU TO DO AND WHY
Analyze these five interview sheets.
Compare each pattern with the desk research and the Calendly analysis:
supports / contradicts / new / still unknown.
Why: I want to see what repeats, where people disagree,
and what fits or contradicts what we already found.

RULES AND WHY
- Say at the top that this is simulated classroom data,
  so no one reads it as real research.
- Give each pattern an ID (P1, P2 …) and link it to an RQ,
  so the dashboard can point to it.
- Keep what people said separate from what you think it means.
- For every pattern, list the participant IDs behind it,
  so I can check where it came from.
- Use real counts instead of "most" or "many",
  because five people is a small number.
- Show where participants contradict each other.
- Name which customer or staff situations are missing from our data.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.
- Stay with findings and leave out solutions and UI ideas,
  because we have not decided what to build.

FORMAT
A table: pattern ID · what they said · who · RQ ·
link to desk research or Calendly · what it might mean
```

8. Dashboard

Where we are desk research, the competitive analysis and the interviews are finished. Now we bring all the findings into one place.

© 2026 Ladan Sojudi

- **What it is:** a one-page summary, made once at the end of Session 2, after all the research is finished: what we are sure of, what is still a hypothesis, and what the next step is.
- **Why we do it:** our findings are spread over desk research, the competitive analysis and the interviews, and we want to see them in one place. We build it only now, because a dashboard drawn earlier would have to be redone after every new finding.
- **The five-second test:** if someone cannot understand in five seconds what we know, what we do not know and what the next step is, the dashboard has not done its job.
- **What goes in it:** a question that got an answer moves to "we know" with a new K, and its earlier RQ stays beside it. A hypothesis we no longer believe is not deleted; it is marked "Deprecated" with one short reason. External evidence and evidence from our own users stay separate.
- **How we do it:** first the skeleton, which we read and approve, then the full dashboard (an Artifact).
- An Artifact is not added to Context; we save its text as a file named "Dashboard v1" and add that to Context.

Skeleton

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook is a beauty salon with three branches.
Customers book hair and nail services, mostly by phone and WhatsApp.
Receptionists take booking calls and serve walk-in customers.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
"Discovery Brief v2"
"Desk Research v1"
"Competitive Analysis v1"
"Interview Synthesis v1" (simulated classroom data)

WHAT I WANT YOU TO DO AND WHY
Show me only the structure of a project dashboard.
Why: I want to agree on the sections before anything is built,
because changing a structure is cheaper than changing a finished page.

RULES AND WHY
- Wait for my approval before building anything.

FORMAT
Section titles + one line describing each.
```

Full dashboard

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
Only the four approved files above.

WHAT I WANT YOU TO DO AND WHY
Create an interactive artifact called: QuickBook Discovery Dashboard
At the top: version v1.0, today's date, what changed.
Show:
- what we know (K IDs), including each answered RQ as a new K
  (next free number), keeping its RQ ID and the pattern IDs behind it
- desk research findings (E IDs)
- competitive analysis findings (Calendly, screen IDs)
- interview findings (P IDs), labeled as simulated classroom data
- contradictions between the sources
- hypotheses: still open, and no longer believed (marked "Deprecated",
  with one very short reason)
- research questions still open
- the next step
Why: our findings are spread over several files. I want to see
in five seconds what we know, what we don't know yet, and what comes next.

RULES AND WHY
- Keep desk research, competitive analysis and interviews separate,
  because they come from different kinds of sources.
- Use a chart only where the files have real numbers,
  because a chart with invented numbers looks like evidence.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.
- Stay with findings and leave out UI ideas,
  because we have not decided what to build.

FORMAT
Visual first, short, easy to scan.
```

Check and save. Check the output in "03 — Review" with `check-skill`, read it yourself, and only then approve it. Save its text as `Dashboard v1.md`, add it to Context, and run `update-files-skill`.

Sources

- **Claude in Chrome and privacy (Competitive analysis):** https://support.claude.com/en/articles/12902428-use-claude-in-chrome-safely

© 2026 Ladan Sojudi

Skills and Plugins — Session 2

Skills we built ourselves

- **check-skill:** after every AI output, in a separate chat, we call it to check the claims against their sources. The text is at the end of this handout (Appendix: check-skill). The `.zip` file is given separately.
- **update-files-skill:** after each approved file, in the same chat, we say "update project files" and it writes the new rows for `state.md`. The text is at the end of this handout (Appendix: update-files-skill).

Ready-made skills

None in this session.

Ready-made plugins

- **Where to find plugins:** you do not download them one by one from the web. They are inside the Claude app: Customize → Plugins → Discover (in Cowork, open the Cowork tab first), choose the plugin and click Add. Paid plans only. The list you see in Discover can change, so look at it in the app.
- Anthropic also publishes plugins on GitHub, if you want to read their texts first: knowledge-work-plugins. The install steps are in Using plugins in Claude.
- Plugins are optional and do not replace our own skills.

Connectors

None in this session. The Figma connector comes in Session 3.

Templates

- The templates at the end of this handout (README.md and state.md) are generic. Fill in the product name and the other [blank] places for each project.

© 2026 Ladan Sojudi

Session 3

This handout summarizes what you should take with you from Session 3. From the approved Dashboard of Session 2 we reached five decisions: problem, persona, solution, user flow and wireframe. The AI helped, but it did not make any of the decisions.

1. Main points

- Every decision has a very short reason, from the interviews or from desk research.
- What the AI wrote is not approved. It is first checked with `check-skill` in the "03 — Review" chat, then we read it ourselves and approve it.
- The approved files of this session: Problem Statement, Persona, Solution, Flows and Wireframes (all v1).
- Every approved output ends the same way as in Session 2: save it as a file, add it to Context, and run `update-files-skill`.

2. Problem

> **Where we are** the Dashboard is approved and in Context. Now we write the problems.

- **What it is:** the main problems, each in one sentence, with one short reason from the interviews or desk research. Like a doctor: first say what the problem is, then give the medicine.
- **Why we do it:** the problems come first, because personas, the solution and the flows are all chosen from them.
- **How we do it:** open a new chat called "09 — Problem v1" and send this prompt. The dashboard file is in Context, so you do not need to paste anything; the AI reads the approved Dashboard and writes the problems.

© 2026 Ladan Sojudi

```
ROLE
You are a UX researcher helping a product design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking app for a service business.
Its user groups are described in our dashboard.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: "Dashboard v1" (our interviews and desk research).
We do not know: anything the dashboard does not cover.

WHAT I WANT YOU TO DO AND WHY
Write the main problems for QuickBook, because we will choose
personas, a solution and flows from them.

RULES AND WHY
- Describe problems only. Solutions come later, after the personas.
- Maximum 4 problems, so we can discuss each one in class.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
For each problem:
- one sentence
- one very short reason, from our interviews or desk research
```

- After the check with `check-skill` in "03 — Review", ask the AI for the final text in a code block, save it as `Problem Statement v1.md`, add it to Context, and run `update-files-skill`.

3. Persona

> **Where we are** the problems are approved. Now we describe the people we design for.

- **What it is:** a short description of each group of users, written only from our evidence.
- **Why we do it:** the solution, the flows and the screens are designed for these people. We want one persona for each group of users, and for each persona one short reason from the interviews or desk research.
- **How we do it:** open a new chat called "10 — Persona v1" and send this prompt. The AI reads the approved Dashboard from Context. Invented names, ages and quotes are forbidden, because "Sara, 32 years old" does not exist in our evidence.

© 2026 Ladan Sojudi

```
ROLE
You are a UX researcher helping a product design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking app for a service business.
Its user groups are described in the dashboard.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: "Dashboard v1" (our interviews and desk research).
We do not know: anything the dashboard does not cover.

WHAT I WANT YOU TO DO AND WHY
Create a user persona for each user group in our dashboard,
because we will design the solution, flows and screens for them.

RULES AND WHY
- Describe each group by its role, not with an invented name, age
  or quote, because those are not in our evidence.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
For each persona:
- Maximum 6 lines: goal, what is hard, what they do today.
- End with one very short reason, from our interviews and desk research.
```

- After the check with `check-skill` in "03 — Review", ask the AI for the final text in a code block, save it as `Persona v1.md`, add it to Context, and run `update-files-skill`.

4. The persona skill

> **Where we are** the personas are approved. Now we use them to test ideas.

- **What it is:** a skill that looks at something from the persona's point of view, only from the persona file. In Session 2 we met `check-skill` and `update-files-skill`; this is a third one. A skill is an instruction we write once and use whenever needed.
- **Why we do it:** its answer is a hypothesis, not a finding. To know whether the screens work, we must test them with real people.
- **How we do it:** in a chat of this Project, say "check this as the persona". Its text is at the end of this handout (Appendix: persona-skill).

© 2026 Ladan Sojudi

5. Solution

> **Where we are** the problems and personas are approved. Now we look for solutions.

- **What it is:** a "How might we…?" question for each problem, and three solution directions for each question. The question is not an answer yet. A direction is an idea in one sentence, not a page or a button.
- **Why we do it:** before writing the flow we must know which solution's flow we are writing; otherwise the AI silently picks a solution. The choice is ours: for each problem we choose one direction and write a short reason next to it.
- **How we do it:** with these two prompts, in a new chat ("11 — Solution v1"). After the second one we choose the directions ourselves, without the AI. We do not choose a direction that has "no evidence".

How might we questions

```
ROLE
You are a product designer helping a design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking app for a service business.
Its users are the personas in "Persona v1".

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: the approved problems and personas.

WHAT I WANT YOU TO DO AND WHY
Write one "How might we…?" question for each problem,
because a question keeps us open to more than one solution.

RULES AND WHY
Questions only. We look for solutions in the next step.

FORMAT
One line for each question, so they are easy to compare.
```

Three directions for each question

© 2026 Ladan Sojudi

```
ROLE
You are a product designer helping a design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking app for a service business.
Its users are the personas in "Persona v1".

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: the approved problems, personas and the dashboard.
We do not know: whether any direction works for real users.

WHAT I WANT YOU TO DO AND WHY
For each question, give 3 different solution directions.
We will choose one direction per problem ourselves,
so we need real alternatives to compare.

RULES AND WHY
- Describe ideas, not screens or buttons. We design screens after the flows.
- If our evidence does not support a direction, write "no evidence",
  so we can see which ideas are guesses.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
For each direction:
- one sentence
- which persona and which problem it serves
- one very short reason, from our interviews or desk research
```

Check and save. After we choose the directions ourselves, check the result with `check-skill` in "03 — Review", save it as `Solution v1.md`, add it to Context, and run `update-files-skill`.

6. User flow

> **Where we are** the solution directions are chosen. Now we write the steps a user takes.

- **What it is:** the steps a user takes to get their task done; like giving directions. First we write it in words, because we have no screens yet. Then it is drawn in FigJam.
- **Why we do it:** the screens we draw next will come from these steps, and a picture makes missing or extra steps easy to spot.
- **How we do it:** with these prompts, in a new chat ("12 — Flow v1"). The first writes the flows in words; the second draws them in FigJam through the Figma Connector, a bridge that lets the AI work in another app. You only need to connect it once.

© 2026 Ladan Sojudi

Flows

```
ROLE
You are a product designer helping a design team.

ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking app for a service business.
Its users are the personas in "Persona v1".

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: the approved problems, personas and "Solution v1".
We do not know: how real users move through it yet.

WHAT I WANT YOU TO DO AND WHY
Write one flow for each persona, for the main thing that persona
does with QuickBook in the chosen solution,
because the screens we draw next will come from these steps.

RULES AND WHY
- Describe what the persona does, not screens or buttons,
  because we choose screens in the next step.
- Stay inside the chosen solution, because it is already approved.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
For each flow:
- Name it after the persona.
- Numbered steps, maximum 8.
- End with one very short reason, from our interviews or desk research.
```

Drawing the flows in FigJam

```
WHAT I WANT YOU TO DO AND WHY
Draw the approved flows in FigJam,
because a picture makes missing or extra steps easy to spot.

RULES AND WHY
Draw only the approved steps, because the flow is already reviewed.

FORMAT
- One flowchart for each flow, one box for each step.
- Show the unhappy paths as dashed side branches.
```

Check and save. Check the written flows with `check-skill` in "03 — Review", save them as `Flows v1.md`, add it to Context, and run `update-files-skill`.

© 2026 Ladan Sojudi

7. Wireframe

> **Where we are** the flows are approved. Now we draw the screens.

- **What it is:** simple grey drawings of the screens. First the list of pages, then what is on each page, then the drawing. Each page has four states: full, empty, loading, error. We choose one key page and put its four states side by side.
- **Why we do it:** we read and approve the list ourselves, because the AI likes to add extra pages. We draw in grey, because with colour and pictures we think the design is finished. The AI usually builds only the brochure-photo state.
- **How we do it:** with these prompts, in a new chat ("13 — Wireframes v1"). We read and approve the list of screens and their contents before anything is drawn.

List of screens

```
ROLE
You are a product designer helping a design team.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: "Flows v1".

WHAT I WANT YOU TO DO AND WHY
List the screens we need to draw as low-fidelity wireframes.
We approve the list before drawing, so no extra screens slip in.

RULES AND WHY
- Only screens that come from a step in the flows.
- Give the list only. We draw after we approve it.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
One line for each screen: the persona, the step it comes from, and its name.
```

Content of each screen

© 2026 Ladan Sojudi

```
ROLE
You are a product designer helping a design team.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: the approved screen list and "Flows v1".

WHAT I WANT YOU TO DO AND WHY
For each screen on the approved list, write what will be on it.
We agree on the content before drawing, so the drawing only shows layout.

RULES AND WHY
- Keep it short, so we can review it in class.
- Write the content only. We draw after we approve it.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
For each screen:
- the main parts (maximum 4)
- the one thing the persona can do there
```

Drawing the wireframes as an Artifact

```
ROLE
You are a product designer helping a design team.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: the approved screen list and the approved contents.

WHAT I WANT YOU TO DO AND WHY
Draw the approved screens as low-fidelity wireframes,
because we want to check the layout, not the look.
Create an interactive artifact called: QuickBook Wireframes

RULES AND WHY
- Grey boxes with labels only, because colour and images make a draft
  look finished.
- Use placeholder labels instead of final copy.
- Only what is on the approved list.

FORMAT
- One screen for each item on the approved list.
- At the top: version v1.0, today's date, what changed.
```

Four states of one screen

© 2026 Ladan Sojudi

- **What it is:** one key screen drawn four times side by side: Default (the normal screen), Empty (for example no free slots), Loading and Error.
- **Why we do it:** real users also meet these states, not only the ideal one. The AI usually builds only the ideal one, so the other three are where mistakes and missing decisions hide.
- **How we do it:** in the same chat ("13 — Wireframes v1"), after the wireframes are drawn. Choose one key screen and put its name in the brackets. Where we do not know how the business handles a case, the AI writes "ask the company", because that decision is the company's, not ours.

```
ROLE
You are a product designer helping a design team.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We do not know: how the business handles most of these cases.

WHAT I WANT YOU TO DO AND WHY
Take screen [persona + screen name] and create 4 variants side by side:
Default, Empty (no free slots), Loading, Error.
Real users also see these states, not only the ideal one.

RULES AND WHY
- Same layout, grey boxes only, so only the state changes.
- If we don't know how the business handles it, write "ask the company",
  because that is the company's decision, not ours.

FORMAT
- Under each variant: one line on what the user can do next.
- Update the version and "what changed".
```

Check and save. Check the wireframes with `check-skill` in "03 — Review", get the text of the artifact, save it as `Wireframes v1.md`, add it to Context, and run `update-files-skill`.

8. Check before approval

- First check, with `check-skill`: is there a page or a claim that has no reason?
- Second check, with the `persona-skill`: does the persona get stuck in the flow?
- Each valid point goes back to the wireframe chat and we change only that page.
- An Artifact cannot be pasted; first we get its text.
- At the end of the session we ask the AI where it went wrong today, so we can add a rule that prevents the same mistake.

Sources

None in this session.

© 2026 Ladan Sojudi

Skills and Plugins — Session 3

This file lists the skills and plugins that will be useful in Session 3. The persona skill is at the end of this handout, together with the generic project templates.

Skills we built ourselves

- **persona-skill:** tells each persona's reaction only from the approved persona file. Use it when you say "check this as the persona". Its text is at the end of this handout (Appendix: persona-skill), and the `.zip` file is given separately.
- To build or install your own skills, see the official guide: Agent Skills overview.

Ready-made skills

None in this session.

Ready-made plugins

- **Install:** Customize → Plugins → Discover → Add (in Cowork, open the Cowork tab first). Plugins are on all paid plans (Pro, Max, Team and Enterprise). Anthropic also publishes plugins in one repository: knowledge-work-plugins. Install guide: Using plugins in Claude.
- Skill names in the installed version may differ a little, so look at the plugin's skill list. These plugins are optional and do not replace our own skills.

Connectors

- **Figma connector:** lets the AI draw the flows in FigJam (section 6). It is a connector, not a plugin, and you connect it once. Install guide: Setting up the Figma remote MCP server

If you use ChatGPT

- Custom GPTs are scheduled to retire on 2026-12-11, and OpenAI plans to move them to plugins. Dates can vary by account or workspace.
- In the planned migration, a GPT's instructions become a skill inside a plugin.
- You can put the `persona-skill` text inside a plugin the same way.
- Source: Custom GPT retirement and migration FAQ

© 2026 Ladan Sojudi

Session 4

Understanding how digital applications are built

This session is designed for product designers who have no programming experience. We will learn what different applications are, how they work, which technologies are used to build them, and how we can use AI to create our own applications.

The goal is not to become a programmer. The goal is to understand enough to make better design decisions and work confidently with coding agents.

1. Types of Digital Applications

Before building an application, we need to understand what kind of product we are creating.

- **Website:** a collection of pages that people visit in a browser, usually to read information or explore content. Example: Wikipedia.
- **Web Application (Web App):** an interactive application that runs in a browser. Users can perform tasks, create content, or manage information. Examples: Gmail, Google Docs, and QuickBook.
- **Mobile Web:** a website or web app opened in a phone's browser. It does not automatically become a mobile app just because we open it on a phone.
- **Mobile Application:** an application installed on a smartphone or tablet. Examples: WhatsApp and Instagram.
- **Desktop Application:** an application installed on a computer, such as Photoshop or VS Code.
- **Progressive Web App (PWA):** a web application that can offer app-like features, such as installation and some offline functionality.
- **Games:** interactive applications that can run in browsers, on phones, computers, or gaming consoles.
- **Embedded and IoT Applications:** software running inside devices such as smartwatches, televisions, and smart home devices.
- **Command-Line Applications (CLI):** applications controlled through text commands rather than a graphical interface.

Different ways to build applications

- **Native:** built specifically for one operating system, such as iOS or Android.
- **Cross-platform:** built using a shared codebase that supports multiple platforms.
- **Hybrid:** combines web technologies with a native application container.
- **Responsive Web:** a website or web app whose layout adapts to different screen sizes.

Example: We can build QuickBook as a web app with React, or package it as a desktop version using Electron.

© 2026 Ladan Sojudi

2. How Applications Work

Imagine a user opens QuickBook, selects 10:00 AM, and clicks Confirm Booking.

What happens behind the screen?

- **Browser:** the application that opens and displays websites, such as Chrome or Safari.
- **Client:** the user's browser or application that sends requests and receives responses.
- **Server:** a computer or software system that receives requests, processes them, and sends responses.
- **Frontend:** the part of an application that users see and interact with.
- **Backend:** the part that handles behind-the-scenes operations, such as checking bookings or managing accounts.
- **API:** a defined way for different parts of software to communicate. For example, the frontend can use an API to ask the backend to create a booking.
- **Request:** a message sent to ask for information or an action.
- **Response:** the answer returned after a request.
- **Database:** an organized place to store and retrieve information, such as users and bookings.
- **localhost:** a special address that points to our own computer, often used to test an application during development.

Example: Confirming a booking

1. The user selects 10:00 AM in the frontend.
2. The frontend sends a request through an API.
3. The backend checks whether the time is available.
4. If available, the booking is saved in the database.
5. The backend sends a response.
6. The frontend displays a success message.

Our training version of QuickBook uses made-up data, so we can practice without building a real backend.

3. Programming Languages and Web Technologies

A programming language is a way to give instructions to a computer using specific words and rules.

Different languages and technologies are used for different tasks.

© 2026 Ladan Sojudi

Language / Technology	What it does	Common use
HTML	Defines the structure and content of a web page	Web
CSS	Controls colours, fonts, spacing, and layout	Web
JavaScript	Makes interfaces interactive and handles application logic	Web, backend
TypeScript	Adds type checking to JavaScript to catch many mistakes earlier	Web, backend
Swift	A programming language commonly used for Apple applications	iOS, macOS
Kotlin	A programming language commonly used for Android applications	Android
Python	A general-purpose programming language	Backend, automation, AI
SQL	A language used to work with relational databases	Databases
C#	A general-purpose programming language	Desktop, backend, games

HTML and CSS are not general-purpose programming languages. HTML describes structure, while CSS describes presentation.

Example: A booking button

Imagine a blue button labelled "Confirm Booking".

- HTML describes the button and its text.
- CSS makes it blue, rounded, and properly spaced.
- JavaScript defines what happens when the user clicks it.

We can also use TypeScript to help catch mistakes in the JavaScript logic.

4. How Applications Are Built

Writing every part of an application from scratch would take a lot of time. Developers use reusable code, libraries, and frameworks to build applications faster.

- **Library:** reusable code that provides specific functionality. We choose when and how to use it.
- **Framework:** a larger structure that provides conventions and tools for organizing and building an application.

© 2026 Ladan Sojudi

- **React:** a JavaScript library for building interfaces from reusable components.
- **Component:** a reusable piece of an interface, such as a button, card, or booking time slot. Similar to a component in Figma.
- **Props:** information passed into a component. For example, a Time Slot component might receive "10:00 AM" as its label.
- **State:** information an interface remembers while the user interacts with it, such as the currently selected booking time.
- **JSX:** syntax that lets developers describe interface elements using HTML-like markup inside JavaScript or TypeScript.
- **`.tsx`:** a TypeScript file that can contain JSX.
- **Package:** a bundle of reusable code that can be installed in a project.
- **Dependency:** a package that a project needs to run or be developed.

Example: Time slots in QuickBook

We design one Time Slot component and reuse it for 9:00, 10:00, and 11:00.

The component receives its time through Props.

When the user selects 10:00, the State changes.

React updates the interface to highlight the selected time.

This is similar to reusable components and interactive variants in Figma, but it works inside a real application.

5. Where Application Data Is Stored

Applications need to remember information, but not all information is stored in the same place.

- **State:** temporary information used by the interface, such as an open modal or selected time.
- **Local Storage:** a browser feature that allows an application to save small amounts of information on the user's device.
- **Database:** a system for storing and organizing information that needs to be retrieved later.
- **Authentication:** the process of verifying who a user is, such as signing in with an email or Apple account.
- **Business Logic:** the rules that decide how an application should behave, such as preventing two users from booking the same appointment.

Example: Selecting vs saving a booking

Selecting 10:00 AM changes the interface State.

Refreshing the page might reset that selection.

Confirming a real booking requires the application to save the booking somewhere persistent, often in a database through the backend.

© 2026 Ladan Sojudi

These are different operations.

6. Projects, Folders, and Files

An application is usually made of many files organized inside a project folder.

- **Project Folder:** the main folder containing the application's code, assets, and configuration.
- **Source Code:** the instructions developers write to create the application.
- **Assets:** files used by the application, such as images, icons, and fonts.
- **Repository (Repo):** a project tracked by a version control system such as Git.
- **`package.json`:** a file that describes a JavaScript project, its dependencies, and available commands.
- **Markdown (`.md`):** a simple text format used for documentation, instructions, and notes.
- **`README.md`:** a document that explains what a project is and how to work with it.
- **`state.md`:** a document we can create to record project progress, decisions, and unfinished work.
- **Rules File:** a file containing instructions for a coding agent, such as `CLAUDE.md` for Claude Code or `AGENTS.md` for Codex.

Example: QuickBook project

Our QuickBook project contains files for its pages, reusable components, images, dependencies, and instructions.

When the coding agent changes a component file, that change can affect every page using the component.

7. Tools for Building and Running Applications

Writing code is only part of development. We also need tools to manage, run, and prepare the application.

- **Code Editor:** a program used to open and edit code files, such as VS Code.
- **IDE:** an integrated development environment that combines code editing with development tools.
- **Terminal:** a place where we give text commands to the computer instead of clicking buttons.
- **Node.js:** a program that allows JavaScript to run outside a web browser, including on our computer.
- **npm:** a package manager used to install and manage JavaScript packages.
- **Vite:** a tool that provides a development server and helps build web applications.
- **Dev Server:** a local server used while developing an application. It usually updates the browser when files change.

© 2026 Ladan Sojudi

- **Build:** the process of preparing source code and assets into files suitable for running or publishing.

Example: Running QuickBook

1. We open the QuickBook folder in the Code tab of the Claude desktop app.
2. We install the project's dependencies using npm.
3. We start the development server using Vite.
4. We open the local address in our browser.
5. We see QuickBook running.
6. When the coding agent changes the code, the browser usually updates.

8. Saving Versions and Publishing Applications

Building an application and publishing it are two different things.

- **Git:** a version control tool that tracks changes to project files.
- **GitHub:** an online platform for storing and collaborating on Git repositories.
- **Commit:** a saved checkpoint in the project's Git history.
- **Hosting:** a service that provides the infrastructure needed to make an application available online.
- **Deployment:** the process of releasing an application to an environment where it can run.
- **Domain:** a human-readable website address, such as `example.com`.

Example: Publishing QuickBook

While developing, we open QuickBook using localhost.

We use Git to track changes and GitHub to store the repository online.

When the application is ready, we build and deploy it to a hosting service.

Users can then open the published web application through its public address.

9. Working With a Coding Agent

Now that we understand the basic structure of an application, we can use AI to help build one.

- **Vibe Coding:** building software by describing what we want to AI and letting it generate or modify the code.
- **Coding Agent:** an AI system that can inspect project files, write code, make changes, and run development commands.
- **Claude Code:** Anthropic's coding agent.
- **Codex:** OpenAI's coding agent.

© 2026 Ladan Sojudi

- **Error Message:** information shown when a tool or program encounters a problem. It can help identify what went wrong.
- **Debugging:** the process of investigating and fixing problems in an application.
- **Testing:** checking whether the application behaves as expected.
- **Visual Review:** inspecting the rendered interface to check layout, spacing, typography, colours, and other design details.

Example: Working with AI

We ask the coding agent to build the QuickBook booking page.

The agent creates the necessary files and runs the application.

We open the page in a browser and inspect the result.

If the Confirm button does not work, we describe the problem and ask the agent to investigate.

If the spacing looks wrong, we explain the expected design and ask the agent to adjust it.

An application can run without errors and still have design or usability problems. That is why reviewing the experience remains our responsibility.

10. What Is a Tech Stack?

A Tech Stack is the collection of technologies used to build and run an application.

Different projects can use different stacks depending on what they need.

For example, our QuickBook learning project uses:

Technology	Role
React	Builds the user interface
TypeScript	Adds type checking to the application code
Vite	Runs the development server and builds the web app
Node.js	Runs development tools on our computer
npm	Installs and manages packages
Git	Tracks project changes
Coding Agent	Helps us create and modify the application

QuickBook does not need a real backend or database for our initial learning exercise.

Main points

What to take with you from Session 4.

You do not need to memorize all these technologies or learn to write code yourself.

© 2026 Ladan Sojudi

You should understand the difference between an application, a programming language, a library, a framework, and a development tool.

You should also understand how the frontend, backend, and database work together, and why building an interface is different from making a complete application.

In Session 5, we will put these concepts into practice by building QuickBook with a coding agent.

Sources

- **Types of applications, how the web works, languages and APIs (sections 1 to 3):** MDN Web Docs, Learn web development: https://developer.mozilla.org/en-US/docs/Learn_web_development
- **Frameworks, components, Props and State (section 4):** React documentation, Learn React: https://react.dev/learn
- **Version control (section 8):** Pro Git book: https://git-scm.com/book/en/v2
- **Node.js (sections 7 and 10):** https://nodejs.org/en/learn/getting-started/introduction-to-nodejs
- **Vite and TypeScript (sections 7 and 10):** https://vite.dev/guide/ and https://www.typescriptlang.org/docs/
- **Coding agents (section 9):** Claude Code documentation: https://code.claude.com/docs/en/overview

© 2026 Ladan Sojudi

Session 5

All of today's prompts, in the order they run in class. Copy every prompt as it is. Things inside [ ] belong to your own project and you must change them. Do not touch the rest.

- Tool: in Claude, the Code tab with the Local folder option, and in ChatGPT the Codex tool. All prompts work in both.
- Until today the outputs were text and you saved them yourself. Today is the first time the AI creates files itself, and everything it builds is saved on your own computer.

1. Two steps before every prompt

- **Step one:** create a folder on your computer named `quickbook`. Until now your work was in the Project and you had no folder; copy the approved files of Sessions 2 and 3 that you saved yourself into this folder.
- **Step two:** open the folder in the Code tab: desktop app → Code tab → Local folder → that `quickbook` folder. In ChatGPT, instead of the Code tab you go to Codex.
- The app code is built in a subfolder named `app` so it does not mix with the course files.
- The `state.md` created in the `app` folder is updated by the AI. The `state.md` inside Context stays separate.
- **Warning:** do not put the Session 2 interview sheets in this folder. If you did, do not send the AI to read them; they are long and are read every time, the findings and decisions are enough.

2. The setup prompt — runs once

- When: first of all, in a fresh chat inside the `quickbook` folder.
- What to change: fill the lines inside [ ] with your product name and the names of your approved files. At the end of the rules you can add your own rule with its reason.
- This prompt creates the project itself, the rules file, README.md and state.md. It builds no pages.

© 2026 Ladan Sojudi

```
ROLE
You are setting up a new project for a designer who does not write code.

ABOUT THE PRODUCT AND ITS USERS
[QuickBook — a booking app for a service business with three branches.]
[It is a web app that people open in a browser.]
Who the users are is described in my approved files below.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
This folder is my course folder. My approved work from earlier sessions
is in it: [Discovery Brief, Problem Statement,
Persona, Flows, Wireframes].
Read these files before you do anything else, because they hold the
decisions the app must follow.
Leave the interview sheets closed: they are long and hold people's
personal details, and the findings in the approved files are enough.
We do not know anything the approved files do not cover.

WHAT I WANT YOU TO DO AND WHY
Set up the project for the rest of the course. Every future chat in
this folder starts from what you set up now, so set it up carefully.
Choose the tools yourself and tell me in one line what you chose.
Create:
1. The project itself, in a new folder called app, with a dev server I
   can start. Everything you create goes inside app.
2. The rules file that you read automatically at the start of every chat
   in this folder — use the file name your own tool expects, and tell me
   what you named it. Put these rules in it:
   - Read my approved files (listed above) before building any screen, so
     every screen comes from my research and not from a guess.
   - Ask before adding a new library, because each one adds weight and
     more things that can break.
   - Keep styles in one CSS file per component, so I can change the look
     of one part without touching the others.
   - Change only the files I ask about, so nothing I have already checked
     changes behind my back.
   - After every change, update state.md yourself: add a History row,
     correct the Status table, and add a Decisions row if a choice was
     made. This is the one exception to the rule above, because the next
     chat needs to know where we stopped.
   - Before saying something works, start the app and report any errors
     from the terminal, because "it should work" is not the same as "it
     runs". Say plainly that you have not looked at the screen — I check
     that.
   [- add your own rules here, each with its reason]
3. README.md, written from the research: what the product is, who the
   users are, and the list of screens we plan to build.
4. state.md with three tables: Status (what works now), Decisions (date,
   decision, reason), History (date, what changed).

RULES AND WHY
```

© 2026 Ladan Sojudi

```
- Keep every file you create inside the app folder, so my course files
  stay exactly as they are.
- Build no screens yet. This chat is only for the setup; the screens
  come in the next prompt, with my wireframes.
- Write README.md only from my approved files, not from general
  knowledge, because it describes our product and our users.
- If something is not in the information above, say you do not know,
  so we can find it out.

FORMAT
In the header of every .md file write the version and the date yourself,
so I can tell versions apart. Ask me only for the creator name.
When you are done, show me the folder structure and tell me how to start
the app.
```

- After it runs: look at the folder structure and ask how to start the app. The rules file is called CLAUDE.md in Claude and AGENTS.md in Codex.
- Two rules must be in the rules file: one, it updates state.md itself after every change. Two, before it says "it works", it runs the app and states clearly that it has not seen the page; seeing the page is your job.

3. One prompt, all the pages

- When: after setup. We build the complete flow at once, because pages built together share one structure.
- What to change: fill [ ] with your approved files and attach the wireframe pictures to the chat. Keep the time-grid line only if one of your pages is a timetable.
- Before building, the AI writes one line for each page. Read it and change it right there; it is the cheapest moment to change the plan.

© 2026 Ladan Sojudi

```
ABOUT THE PRODUCT AND ITS USERS
QuickBook, a booking web app. The flow, the screens, the states and the
users are described in my approved files.

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
1. [My approved files] — my own research. The flow, the screens and the states
   are described there. Read all of it before anything else.
2. A flow map that lists every step and marks which screens I drew and
   which ones you design. Follow it.
3. The pictures I attached, drawn by me. The flow map says which
   screen each one is. Follow their layout.
   They are grey on purpose — there is no colour direction yet, so
   use plain neutral styling for now.
We do not know what should happen in cases the research does not cover.

WHAT I WANT YOU TO DO AND WHY
Build the whole QuickBook booking flow in one go. When all the screens
are built together, they share one structure. Screens added one by one
each get a new decision and stop looking alike.
1. Every screen in the flow and every state the research names.
   For the screens I drew, follow my pictures.
   For the screens I did not draw, design them yourself from the research.
2. Go through the whole flow as if you were using it, and list the cases
   the research does not cover — what happens when something fails, when a
   name is longer than the card, when I click twice, when I come back an
   hour later.
   For each one: the case in one line, what you propose, and whether it
   comes from my research or is your own suggestion, so I can see where you
   used my research and where you guessed.
   Order by how likely it is. Maximum 6. Build none of them yet, because I
   choose which ones go in.
3. A floating bar, pinned to the bottom centre of the window, always
   on top of the page, so I can show every state without clicking through
   the flow.
   - One button per screen and state, with its name on it.
   - The one I am looking at is clearly marked.
   - Clicking switches immediately, no reload.
   - Big enough to read on a projector.
   This bar is a tool for me, not part of the product: keep it in one file
   called StateSwitcher and give it its own plain look, unlike the product,
   so nobody mistakes it for part of the app and I can remove it in one
   step. Tell me the one line I delete to remove it.

RULES AND WHY
- Let the screens hand over to each other, without a routing library,
  because for a few screens a library only adds complexity.
- What the customer picked travels with them, so nothing is asked twice.
- One folder per screen, with its own .tsx and .css, so each screen can
  be changed on its own.
- Fake data in one file called data.ts, so I can change the examples in
  one place.
```

© 2026 Ladan Sojudi

```
- Show every state with a word or a mark as well as colour, because not
  everyone can tell colours apart.
- Ask before adding any library, because each one adds things that can
  break.
- Keep the page working when the window gets narrower: a maximum width
  on the page, and things wrap instead of overflowing. Use flexible
  widths on the page itself, not fixed pixel widths, because people
  open it on screens of every size.
- Design for the desktop window only for now. I will ask for a phone
  version separately, after I have decided its layout myself.
[- If one of my screens is a time grid, build it as a single CSS
  grid: a time column plus one column per person, one row per
  30 minutes, and a booking spans rows — 30 min is 1 row, 60 min is
  2 rows — so a longer service takes more room on the grid.
  Delete this line if your project has no time grid.]
- If something is not in the information above, say you do not know,
  so we can find it out.

FORMAT
Before you build, give me one line per screen: which screens you are
going to make, and for the ones I did not draw, what goes on them and
what you decided to leave out. Then wait for me, because this is the
cheapest moment to change the plan.
```

- Read the page map. If something is missing or extra, say so right there in the chat and then give permission to build.
- The AI lists at most six unexpected cases. Choose the ones you want and build them with this prompt:

```
WHAT I WANT YOU TO DO AND WHY
Build cases 1, 3 and 5 from your list and leave the rest for now.
Add them to the StateSwitcher too, so I can see each one.

RULES AND WHY
- Keep everything that already works as it is, because I have already
  checked it.

FORMAT
One line for each case: what you built.
```

4. Design system — choose one

- When: after the flow is built and works.
- **Important:** none of these ways is 100% enough. The AI may not do exactly what you asked, so read the file it writes and fix what is wrong. In the end, all that matters is one file in your folder: `design-system.md`.

© 2026 Ladan Sojudi

- You can get that file in three ways: from a published system (way one), from your own design system in Figma (way two), or from an open-source design system whose rules are already written as `.md` files (way three).
- **Way one — from a published system** (Material Design 3): the AI reads only the foundations from the official documentation and writes `design-system.md`. Change [ ] if you want another system.

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The source is the official documentation of [Material Design 3]:
[https://m3.material.io]
Read it from there, not from your memory, because remembered numbers
are often wrong.

WHAT I WANT YOU TO DO AND WHY
I want a design system for this project based on [Material Design 3],
so every screen uses one set of named values instead of guesses.
Take only the foundations — colour roles, the type scale, the spacing
scale and the corner radii. Leave out their components, their icons,
their illustrations and their brand colours, so the product looks like
ours and not like theirs.
Then write design-system.md into this folder.

RULES AND WHY
- Every value has a name, and next to it where you read it, so I can
  check where each number came from.
- If you cannot read a value from the page, say so and mark the one you
  chose as your own, so I know which values to check.
- The typeface may not be free to use, and this will be published on
  the web. If it is not free, pick a free one with the same proportions
  and tell me why you picked it.
- Six to eight spacing values and three radii, so the system stays small
  enough to remember.

FORMAT
A markdown file. One table per group: token, value, source, where to
use it. At the end: which values you read, and which you chose.
```

- **Way two — from your own design system in Figma:** the AI reads the variables and styles in your Figma file (this needs the Figma connection from Session 3) and writes `design-system.md`. Put the link of your Figma file in place of [ ].

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The source is my own design system in Figma: [link to your Figma file].
Read the real variables and styles from the file with the Figma
connection, not from a screenshot, because the values in the file
are exact. Read:
- the colour variables and colour styles
- the text styles: font family, size, weight and line height
- the spacing and radius values used in the components
If the file has no status colours, say so.

WHAT I WANT YOU TO DO AND WHY
Write design-system.md into this folder yourself, from the values you
read, so every screen can use one set of named values. Put in it:
- Colour: brand colour, page and card surfaces, borders, text colours
  (primary, secondary, disabled), and status colours for success,
  warning and error.
- Type: one font family, and the sizes with their weight and line height.
- Spacing: one scale of six to eight numbers.
- Radius: three values — small, medium, large.

RULES AND WHY
- Use the values you actually read, and keep my Figma names, so I can
  find each value in Figma.
- Where something is not in the file, build it from what is, and say
  that you did, so I know which values are real.
- Take the foundations only: leave out components and layouts, because
  the screens are built from my wireframes.

FORMAT
A markdown file. One table per group: token, value, where to use it.
At the end, two short lists: which values you read, and which ones you
chose yourself.
```

- **Way three — from an open-source design system:** find an open-source design system that publishes its rules as `.md` files, put that file in your folder, and ask the AI to keep only the foundations. We do not copy the look of a website, because the screens must be ours.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
I added [file-name].md to this folder. It comes from an open-source
design system: [link to the source].
We do not know if every value in it is complete or fits QuickBook.

WHAT I WANT YOU TO DO AND WHY
Read that file and write design-system.md from it, so every screen
uses one set of named values. Keep only the foundations: colour, type,
spacing and radius. Leave out components, icons and illustrations, so
the product looks like ours.

RULES AND WHY
- Use only values that are in the file. Where something is missing
  (for example status colours), build it from what is there and mark it
  as your own choice, so I can check it.
- Every value has a name, so every screen can use it by name.
- Tell me what the licence of the source allows, because this will be
  published on the web.

FORMAT
A markdown file. One table per group: token, value, where to use it.
At the end, two short lists: which values you read, and which ones you
chose yourself.
```

5. Applying the design system

- When: once you have put `design-system.md` in the `quickbook` folder.
- Be sure to read the last two lists: the places where the file was not enough.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
I have added design-system.md to the project folder.

WHAT I WANT YOU TO DO AND WHY
Apply it to every screen that already exists, so every screen gets its
look from one file and they all match.
- Turn every value in the file into a CSS variable in global.css.
- Replace every colour, size, spacing and radius in the components with
  those variables.

RULES AND WHY
- Keep the content, the flow and the behaviour as they are, because this
  step changes only the look.
- Use only values from design-system.md, so the look has one source.
- Where a screen needs something the file does not have, stop and ask
  me, because a value you invent quietly is one I never get to check.
- Show every state with a word or a mark as well as colour, because not
  everyone can tell colours apart.

FORMAT
When you are done, two short lists, so I can see where the file was not
enough:
1. every value you had to ask me about or invent
2. every place where the old value did not map to anything in the file
```

6. Four prompts — pick one based on your situation

- **Layout and spacing are not right — the most important prompt today.** This is where your work separates from everyone else's. Fill [ ] with what you see with your own eyes and your own numbers.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The content and the flow of [this screen] are right. The layout is not.
The design system gave colour, type and spacing, but not the layout.
What is wrong now:
[What you see with your own eyes. Example:]
- The content sits against the edges of the window and stretches as
  wide as the screen allows.
- Everything is crammed into the top third and the rest is empty.
- The buttons stretch to fill the width, so each one is far bigger than
  the text inside it.

WHAT I WANT YOU TO DO AND WHY
Rework the layout and spacing of [this screen] with the numbers below,
because I am giving the layout now.
Layout:
[Your own numbers. Example:]
- A container, maximum width 1200px, centred, 32px of air each side.
- Two columns: the main side, and a 340px summary column beside it.
- Any tappable cell at least 44px tall, and no wider than 160px – 44px
  is the smallest touch target Apple recommends.
- 32px between sections, 16px between cards, 24px inside a card.
  Use only these vertical numbers, so the rhythm stays even.
Responsive:
- One breakpoint at 900px. Above it, the layout above. Below it, one
  column: the summary moves under the main content and stops sticking
  to the top.
- Below 900px use flexible widths only, and the container padding drops
  from 32px to 16px.
- Any tappable cell stays at least 44px tall at every width.
- Tell me which parts cannot work in one column and what you suggest
  instead. Suggest a change to the existing screen rather than a new
  phone screen, because the phone layout is my decision.

RULES AND WHY
- Use only design-system.md, so the look keeps one source.
- Keep the content, wording and behaviour as they are, because this
  step is only about layout.

FORMAT
Tell me which numbers you used for the container, the columns and the
grid. Then tell me what changes at 900px, so I can check it against
what I asked.
```

- **You do not like the whole look:** you get three different directions for the same page and switch between them with the StateSwitcher.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
The content of [this screen] is right, the look is not.

WHAT I WANT YOU TO DO AND WHY
Give me three different UI directions for the same screen — same
content, same flow, same design system — because I want to choose
between options rather than accept one guess.
Build all three and let me switch between them with the StateSwitcher.

RULES AND WHY
- Use only design-system.md, so all three stay in our system.
- Make them genuinely different, not three paddings of one layout, so
  the choice is a real one.

FORMAT
For each one, one line: what you made bigger, what you grouped, what
you took away.
```

- **You know it is not good but not where:** you get an ordered list, with no changes:

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
[This screen] works and it uses my design system. Judge it against:
1. design-system.md — is every value really coming from it?
2. [My approved files] — does the screen still serve the flow described there?
3. A first-time customer in a hurry — what is unclear or easy to get
   wrong.

WHAT I WANT YOU TO DO AND WHY
Tell me where it could be better, so I can choose what to improve.
Use the state switcher and look at every state, not only the normal
one, because problems often hide in the rare states.

RULES AND WHY
- Give me the list only and make no changes yet, because I choose what
  to apply.
- Maximum 8 items, so we can discuss each one.
- If something is fine, say so, so I leave it alone.
- For each claim, say which part of the information above it comes
  from, so we can check it.
- Use only the information above, because we have no other data.

FORMAT
A numbered list, most important first. For each: which state, what is
weak, what you would change, why it is better in one sentence, and the
size of the change — small / medium / rethink.
```

7. When something breaks

- **You saw an error:** copy the whole error message, not a summary of it.

© 2026 Ladan Sojudi

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
I ran the app and got this error. Here is the full message:

[paste the whole error here]

WHAT I WANT YOU TO DO AND WHY
Tell me in one sentence what caused it, so I learn to recognise it next
time. Then fix it.

RULES AND WHY
- Change only what the error needs, because everything else is
  already checked.

FORMAT
One sentence for the cause, then the fix.
```

- **Still not right after two fixes in a row:** send this, and it is better to continue in a fresh chat:

```
WHAT I WANT YOU TO DO AND WHY
Revert the last change and tell me what you would try differently,
because two fixes in a row have not worked.

RULES AND WHY
- Change nothing else until I answer, because I choose the next try.

FORMAT
Two short parts: what you reverted, and what you suggest instead.
```

- **No error but the page is wrong:** describe what you see:

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
No error, but the screen is wrong. Here is what I see:
[describe it: the blocks overlap / the columns collapsed into one /
the summary card is under the grid instead of beside it]

WHAT I WANT YOU TO DO AND WHY
Fix only that, so nothing else changes.

RULES AND WHY
- Keep the content and the wording as they are, because they are
  already right.
```

8. Skills

- Skills are optional and are not needed to follow the session.
- Their list and install method are in the "Skills and Plugins" part after this session.

© 2026 Ladan Sojudi

Sources

- **Design system (section 4):** https://m3.material.io (Material Design 3 documentation)
- **Ready-made skills:** https://github.com/anthropics/skills/tree/main/skills

© 2026 Ladan Sojudi

Skills and Plugins — Session 5

Skills we built ourselves

- We have no skill of our own for Session 5; the `check-skill` and `update-files-skill` from Session 2 can also be used in this session. Their texts are in the Appendix.

Ready-made skills

- **`frontend-design` skill** (built by Anthropic): takes the look of the pages away from "looks like every site built with AI". It helps when you do not like the look; in the prompt tell it to use only `design-system.md`.
- **`webapp-testing` skill** (built by Anthropic): opens the app in the browser, clicks, and takes screenshots of the page. It helps when you want the page really seen before "it works". It has Python scripts and needs Playwright.
- **Install rule:** install only from an official source and read its description page before installing, because skills can run commands on your computer.
- **Installing both:** send this prompt in the Code tab. The skills sit inside the `quickbook` folder and are on only for this project.

```
EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
Both skills are in Anthropic's official repository:
https://github.com/anthropics/skills/tree/main/skills

WHAT I WANT YOU TO DO AND WHY
Add two skills to this project, so you can use them in every
future chat in this folder: frontend-design and webapp-testing.
Copy each skill folder into .claude/skills/ in this folder.

RULES AND WHY
- Copy only from that repository, because a skill can run code
  on my computer.
- Keep the skill files exactly as they are, so they keep working
  the way Anthropic wrote them.
- Tell me if a skill needs something installed first, so I can
  decide before you install it.

FORMAT
One line per skill: where you put it, and what it needs.
```

Connectors

None in this session.

© 2026 Ladan Sojudi

Appendix — Templates and Skill Files

This part holds the texts that the sessions refer to. Copy a text, save it as a file with the name in its heading, and add it to Context. The `.zip` skill files are given separately.

Project instructions template

Use this empty template for any other project. Paste it into the Project's Instructions (not into Context) and fill in the [blank] places.

```
ROLE
You are helping [who] with [project name]. Before any work, read
README.md and state.md, because they hold our rules and where the
project is now.

RULES AND WHY
- [Rule 1], because [reason].
- [Rule 2], because [reason].
- Write a guess as a guess, never as a fact.
- Say "I don't know" when our files do not answer something.
- Give a source for every important claim, so I can check it.
```

README.md template

Fill in the [blank] places for your product.

© 2026 Ladan Sojudi

```
# [Product name] — README

Version: v1.0 · Date: [date] · Created by: [name]

Read this first. It holds the rules of this Project.
Where the project is right now lives in state.md.

## 1. Project

[Product name] is [one sentence: what it is and who uses it].
The client asked for [the request in one sentence].
Our current job is [the current job, for example: find where the problem really is,
not design].

## 2. Files in Context

- Only approved files live in Context. Drafts stay in the chat.
- One version of each file. When a new version is approved, the old one is deleted
from Context.
- Files are Markdown (.md), short, one topic per file.
- No names, phone numbers or full interview transcripts.

## 3. Naming

- Approved files: `Name vN.md`
- Chats: `NN — Topic vN` (for example `02 — Desk Research v1`). One task per chat.
- Ideas that do not belong to the current task go to the chat `Parking Lot`.
- Every file starts with: `Version · Date · Created by`.

## 4. IDs

Keep only the ID types you use. Add a type here the first time you use it.

| ID | Meaning | Example |
|---|---|---|
| K | Something we know | K3 |
| RQ | Research question: something we must find out | RQ2 |
| S | Source of external research | S1 |
| E | Finding from external research | E04 |
| H | Hypothesis (a guess, never a fact) | H1 |

- IDs never change and are never reused.
- When an RQ is answered, the answer becomes a new K and keeps its RQ as a reference.

## 5. Evidence

- Keep evidence, interpretation, guesses and unknowns separate.
- External research is evidence about other people, not about our users.
- Mark simulated or practice data clearly.
```

© 2026 Ladan Sojudi

state.md template

Fill in the [blank] places for your product.

```
# [Product name] — state

Version: v1.0 · Date: [date] · Created by: [name]

Where the project is right now. Read it at the start of every chat.
Keep it short: it is a summary, not a diary.

## 1. Status

| Item | Value |
|---|---|
| Current phase | [phase] |
| Approved files | README.md (v1.0); [other approved files] |
| Open research questions | [RQ IDs] |
| Next step | [next step and the chat name for it] |

## 2. Decisions

| Decision | Why | What we rejected |
|---|---|---|
| Only approved, short files live in Context | AI cannot tell which of two versions is right | Drafts and old versions |
| One task per chat | Short chats are clearer and use fewer tokens | One long chat for everything |

## 3. Open questions

| Question | Why it is open |
|---|---|
| [question] | [reason] |

## 4. History

| Version | Date | Created by | What changed | Why |
|---|---|---|---|---|
| v1.0 | [date] | [name] | Project created with README.md and state.md | Start of the project |
```

© 2026 Ladan Sojudi

Skill file: check-skill.md

```
Name: check-skill
Use when I ask you to review or check an output.

Review this output before I approve it.

1. Check it against the approved project context.

2. Keep these separate:
- evidence
- interpretation
- assumption
- unknown

3. For every important factual or research claim:
- identify the exact claim
- identify the original source
- check whether the source supports the exact claim

4. Flag:
- unsupported claims
- invented information
- contradictions
- missing evidence
- assumptions presented as facts

5. When relevant, label claims as:
- Verified
- Partially supported
- Not supported
- Not relevant

Do not silently rewrite the output.
Show me what needs human review.
```

© 2026 Ladan Sojudi

Skill file: persona-skill.md

```
Name: persona-skill
Use when I ask you to check something "as the persona".

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: "Persona v1", based on our evidence.
We do not know: anything the persona file does not say.

WHAT I WANT YOU TO DO AND WHY
When I ask you to check something "as the persona",
say how each persona would likely react,
so we can test an idea before we test it with real people.
1. Read what I give you.
2. For each persona, say how this persona would likely react.
3. For each point, quote the line in the persona file it comes from.
4. If the file does not say, write "not in persona".

RULES AND WHY
- Use only what the persona file says, because quotes, names, ages
  and feelings that are not in the file are inventions, not evidence.
- If something is not in the information above, say you do not know,
  so we can find it out.
- For each claim, say which part of the information above it comes from,
  so we can check it.
- Use only the information above, because we have no other data.

FORMAT
End with one line: this is a hypothesis
to test with real people, not a finding.
```

© 2026 Ladan Sojudi

Skill file: update-files-skill.md

```
Name: update-files-skill
Use when I say "update project files".

EVIDENCE WE HAVE AND WHAT WE DO NOT KNOW
We have: README.md and state.md in Context,
and the file I just approved in this chat.
We do not know: anything else about the project.

WHAT I WANT YOU TO DO AND WHY
Update state.md after an approved file,
so the next chat knows where we stopped.
1. Status table: add the approved file, and correct
   the current phase, open research questions and next step.
2. Decisions table: add a row only if a choice was made,
   with the reason.
3. History table: add a row with version, date,
   created by, what changed and why.
4. Say whether README.md needs a change (a new ID type,
   a new file name). Do not change it unless I say so.

RULES AND WHY
- Change only state.md, because every other file is approved.
- You cannot put files in the Project, so show the full
  updated state.md in one code block for me to save.
- If something is not in the information above, say you do not
  know, so we can find it out.
- Use only the information above, because we have no other data.

FORMAT
The full updated state.md in one code block,
then one line: what changed.
```

© 2026 Ladan Sojudi